

Fiche 306 — Listes et data frames : la structure qui porte tout le reste

Matière	Code · Langage R
Cours source	R Core Team, <i>An Introduction to R</i> 4.6.1 — chapitre 6 « Lists and data frames » (§6.1 listes, §6.2 construire et modifier, §6.2.1 concaténer, §6.3 data frames, §6.3.1 les fabriquer, §6.3.2 <code>attach()</code> et <code>detach()</code> , §6.3.3 méthode de travail, §6.3.4 listes quelconques, §6.3.5 le chemin de recherche)
Sources d'appoint	<i>R Language Definition</i> 4.6.1, §2.1.2 « Lists », §2.3.2 « Data frame objects », §3.4.3 « Indexing other structures »
Difficulté	Fondamental — la liste est ce que R rend quand il rend un résultat
Temps d'étude estimé	1 h 15
Prérequis	Fiches 301 à 305
Concepts clés	liste, composante, <code>[[</code> contre <code>[</code> , <code>\$</code> et l'appariement partiel, <code>Lst[[x]]</code> avec nom calculé, <code>length()</code> au niveau supérieur , copie à la construction, extension par indice, <code>c()</code> sur des listes, data frame = liste de classe <code>"data.frame"</code> , les trois restrictions , <code>data.frame()</code> , <code>as.data.frame()</code> , <code>attach()</code> / <code>detach()</code> , masquage , position 2 , chemin de recherche, <code>search()</code> , <code>ls(2)</code> , <code>.GlobalEnv</code>
À retenir en priorité	<code>Lst[[1]]</code> ≠ <code>Lst[1]</code> · un data frame est une liste · <code>attach()</code> attache une copie · <code>u <- v+w</code> ne modifie pas la colonne · le chemin de recherche .

Vue d'ensemble

UNE LISTE	une collection ORDONNEE d'objets appeles COMPOSANTES « THERE IS NO PARTICULAR NEED FOR THE COMPONENTS TO BE OF THE SAME MODE OR TYPE »
TROIS ACCES	<code>Lst[[3]]</code> par NUMERO -- toujours possible <code>Lst[["nom"]]</code> par nom, en chaine -- CALCULABLE <code>Lst\$nom</code> par nom, littéral -- NON calculable

LA DISTINCTION QUI COMPTE

`Lst[[1]]` L'OBJET lui-meme, le nom n'est PAS inclus
`Lst[1]` une SOUS-LISTE d'une entree, les noms sont TRANSFERES

ABREVIATION `Lst$coefficients` peut s'ecrire `Lst$coe`
(appariement partiel -- pratique en console, dangereux en script)

CONSTRUIRE `list(nom1 = obj1, ...)` les composantes sont COPIEES

ETENDRE `Lst[5] <- list(matrix = Mat)`

CONCATENER `c(listeA, listeB)` -> une liste ; les autres attributs (dim) SAUTENT

DATA FRAME « A data frame is A LIST WITH CLASS "data.frame" »
3 restrictions : types admis, colonnes fournies, MEME LONGUEUR

CHEMIN DE RECHERCHE

`.GlobalEnv` -> [attaches] -> Autoloads -> package:base
`attach(df)` place df en POSITION 2 -- et c'est une COPIE
`u <- v+w` ne modifie PAS `df$u` : il le MASQUE en position 1

Le problème posé. « *An R list is **an object consisting of an ordered collection of objects known as its components**. There is no particular need for the components to be of the same mode or type — a list could consist of a numeric vector, a logical value, a matrix, a complex vector, a character array, **a function**, and so on.* » (§6.1)

INTUITION — POURQUOI LA LISTE EST LA STRUCTURE LA PLUS IMPORTANTE DE R.

Le §1.3 disait que R « *will give minimal output and **store the results in a fit object*** » (fiche 300). Cet objet de résultat **est presque toujours une liste** : `eigen()` rend une liste de deux composantes, `svd()` une liste de trois, `lsfit()` une liste, `lm()` une liste. **Savoir lire une liste, c'est savoir lire tout résultat R.** Et un data frame — la structure de données centrale de l'analyse — **est une liste**, avec une classe par-dessus.

● Concept 1 — La liste et ses trois accès

L'exemple du cours (§6.1) :

```
Lst <- list(name = "Fred", wife = "Mary", no.children = 3,
```

```
child.ages = c(4, 7, 9))
```

Règle (§6.1). *« **Components are always numbered and may always be referred to as such.**

Thus if `Lst` is the name of a list with four components, these may be individually referred to as `Lst[[1]]`, `Lst[[2]]`, `Lst[[3]]` and `Lst[[4]]`. If, further, `Lst[[4]]` is a vector subscripted array then `Lst[[4]][1]` is its first entry. »*

« If `Lst` is a list, then **the function** `length(Lst)` gives the number of (top level) components it has. »

« **Top level** » n'est pas un détail. `length(Lst)` vaut **4**, alors que la liste contient sept valeurs élémentaires. La longueur d'une liste compte **ses composantes**, pas ce qu'il y a dedans — la liste est **récursive** (fiche 303).

Règle (§6.1). *« Components of lists **may also be named**, and in this case the component may be referred to **either by giving the component name as a character string in place of the number in double square brackets**, or, more conveniently, by giving an expression of the form `name$component_name`. This is a very useful convention as it makes it easier to get the right component if you forget the number. »*

La table d'équivalence du cours :

Écriture par nom	Écriture par numéro	Valeur
<code>Lst\$name</code>	<code>Lst[[1]]</code>	la chaîne <code>"Fred"</code>
<code>Lst\$wife</code>	<code>Lst[[2]]</code>	la chaîne <code>"Mary"</code>
<code>Lst\$child.ages[1]</code>	<code>Lst[[4]][1]</code>	le nombre 4

MÉTHODE — LE CAS DU NOM CALCULÉ (§6.1).

« Additionally, one can also use **the names of the list components in double square brackets** : `Lst[["name"]]` is the same as `Lst$name`. This is especially useful when the name of the component to be extracted is stored in another variable, as in :

```
x <- "name"; Lst[[x]]
```

» — c'est exactement la règle de la fiche 302 : **\$** n'évalue pas son index, **[[** oui. Dès qu'un nom vient d'une variable, **[[** est le seul outil correct.

1.1 **[[** contre **[** — la distinction fondamentale

Règle (§6.1). « *It is very important to distinguish `Lst[[1]]` from `Lst[1]` . `[[...]]` is the operator used to select a single element, whereas `[...]` is a general subscripting operator. Thus the former is the first object in the list `Lst` , and if it is a named list the name is not included. The latter is a sublist of the list `Lst` consisting of the first entry only. If it is a named list, the names are transferred to the sublist.* »

	<code>Lst[[1]]</code>	<code>Lst[1]</code>
Ce qu'on obtient	l'objet lui-même	une sous-liste d'une entrée
Classe du résultat	celle de l'objet (<code>character</code>)	<code>list</code>
Le nom	n'est pas inclus	est transféré à la sous-liste
<code>length()</code> du résultat	celle de l'objet	1

C'est la source d'erreur numéro un sur les résultats de fonctions. `eigen(Sm)[1]` rend **une liste** contenant les valeurs propres ; `eigen(Sm)[[1]]` rend **le vecteur** des valeurs propres. La première ne peut pas être additionnée, tracée, ni passée à une fonction numérique — et le message d'erreur ne parle pas de crochets.

1.2 L'abréviation des noms

Règle (§6.1). *« The names of components may be abbreviated down to the minimum number of letters needed to identify them uniquely. Thus `Lst$coefficients` may be minimally specified as `Lst$coe` and `Lst$covariance` as `Lst$cov` . »*

C'est l'appariement partiel de la fiche 302, vu du côté du confort. **Il est commode en console et dangereux en script** — la *R Language Definition* rappelait que `$` apparie **en silence**, que `[[` peut avertir (`exact = NA` , son défaut), et surtout que **l'appariement partiel ne joue pas en écriture** : `Lst$coe <- 0` **crée une composante** `coe` à côté de `coefficients` .

REMARQUE (§6.1).

« *The vector of names is in fact simply an attribute of the list like any other and may be handled as such. Other structures besides lists may, of course, similarly be given a `names` attribute also.* » — cohérent avec la fiche 303 : `names` n'a rien de propre aux listes.

● Concept 2 — Construire, étendre, concaténer

DÉFINITION (§6.2).

« New lists may be formed from existing objects by the function `List()` . An assignment of the form

```
Lst <- list(name_1 = object_1, ..., name_m = object_m)
```

sets up a list `Lst` of *m* components ... giving them names as specified by the argument names (which can be freely chosen). If these names are omitted, the components are numbered only. »

Le point le plus important du paragraphe (§6.2). « *The components used to form the list are copied when forming the new list and the originals are not affected.* »

C'est la sémantique de copie de R (fiche 302), énoncée ici explicitement. Mettre un objet dans une liste **ne crée pas de lien** avec l'original : modifier l'un ne modifie pas l'autre. La seule exception du langage reste **l'environnement** (fiche 303).

Étendre (§6.2). « *Lists, like any subscripted object, can be extended by specifying additional components* » :

```
Lst[5] <- list(matrix = Mat)
```

C'est le mécanisme du §3.2 (fiche 303) : écrire hors des bornes allonge l'objet.

DÉFINITION (§6.2.1).

« When the concatenation function `c()` is given list arguments, the result is an object of mode `list` also, whose components are those of the argument lists joined together in sequence. »

```
list.ABC <- c(list.A, list.B, list.C)
```

« Recall that **with vector objects as arguments** the concatenation function similarly joined together all arguments into a single vector structure. **In this case all other attributes, such as `dim` attributes, are discarded.** » — la remarque du §5.9 (fiche 305) : `c()` efface `dim`.

Rappel de la fiche 301. La note 1 du §2.1 prévenait déjà : « With other than vector types of argument, such as **list mode arguments, the action of `c()` is rather different** ». Voici la précision annoncée : sur des listes, `c()` **concatène les composantes** — il n'imbrique pas et n'aplatit pas les composantes elles-mêmes.

● Concept 3 — Le data frame

DÉFINITION (§6.3).

« **A data frame is a list with class `"data.frame"`.** »

Une phrase, et tout est dit. Le reste du paragraphe énumère **les restrictions** qui font qu'une liste peut devenir data frame.

Règle — les trois restrictions (§6.3).

1. « *The components must be vectors (numeric, character, or logical), factors, numeric matrices, lists, or other data frames.* »
2. « *Matrices, lists, and data frames provide as many variables to the new data frame as they have columns, elements, or variables, respectively.* »
3. « *Vector structures appearing as variables of the data frame must all have the same length, and matrix structures must all have the same number of rows.* »

EN CLAIR.

La restriction 3 est **la** contrainte : toutes les colonnes ont **la même longueur**. C'est ce qui autorise la lecture matricielle. La restriction 2 est la plus surprenante : une **matrice** placée dans un data frame y fournit **autant de variables qu'elle a de colonnes** — elle n'y entre pas comme une seule colonne.

Règle (§6.3). « *A data frame may for many purposes be regarded as a matrix with columns possibly of differing modes and attributes. It may be displayed in matrix form, and its rows and columns extracted using matrix indexing conventions.* »

« **For many purposes** », pas pour tous. La fiche 302 a donné l'exception : « *if a single index is supplied, it is interpreted as indexing the list of columns* » (R Language Definition §3.4.3). `df[2]` est **la deuxième colonne**, `df[2, ]` la deuxième **ligne**. Le data frame est matriciel **quand on lui donne deux indices** et redevient **une liste** quand on n'en donne qu'un.

Le construire (§6.3.1).

```
accountants <- data.frame(home = statef, loot = incomes, shot = incomef)
```

*« A list whose components conform to the restrictions of a data frame may be **coerced into a data frame using the function** `as.data.frame()` . »*

« *The simplest way to construct a data frame from scratch is to use the `read.table()` function to read an entire data frame from an external file.* » — fiche 307.

COMPLÉMENT (R LANGUAGE DEFINITION §2.3.2).

« Data frames are the R structures which **most closely mimic the SAS or SPSS data set**, i.e. a **"cases by variables" matrix of data**. ... In addition, a data frame generally has a `names` attribute labeling the variables and a `row.names` attribute for labeling the cases. »

● Concept 4 — `attach()` et `detach()` : commodité et piège

Le besoin (§6.3.2). « The `$` notation, such as `accountants$home`, for list components is **not always very convenient**. A useful facility would be somehow to **make the components of a list or data frame temporarily visible as variables under their component name**, without the need to quote the list name explicitly each time. »

```
attach(lentils)      # lentils a trois variables : u, v, w
```

DÉFINITION (§6.3.2).

« `attach()` **places the data frame in the search path at position 2**, and **provided there are no variables `u`, `v` or `w` in position 1**, `u`, `v` and `w` are **available as variables from the data frame in their own right**. »

4.1 Le piège du masquage

PROBLÈME (§6.3.2).

« At this point an assignment such as `u <- v + w` **does not replace the component `u` of the data frame**, but rather **masks it with another variable `u` in the workspace at position 1 on the search path**. »

« To make a permanent change to the data frame itself, the simplest way is to resort once again to the `$` notation : `lentils$u <- v + w`. However the new value of component `u` is not visible until the data frame is detached and attached again. »

C'est un double piège, et le second est pire que le premier.

Écriture	Ce que ça fait	Ce qu'on croit qu'elle fait
<code>u <- v + w</code>	crée un <code>u</code> dans l'espace de travail, qui masque la colonne	modifier la colonne
<code>lentils\$u <- v + w</code>	modifie la colonne — mais le <code>u</code> visible reste l'ancien	modifier, et voir le changement

Après `lentils$u <- v + w`, un `u` tapé à la console rend l'ancienne valeur : la copie attachée en position 2 n'a pas bougé. Il faut `detach()` puis `attach()` pour que le changement devienne visible. Deux façons d'écrire, deux façons de se tromper.

4.2 Détacher

Règle (§6.3.2). « *To detach a data frame, use the function `detach()`. More precisely, this statement detaches from the search path the entity currently at position 2. ... Entities at positions greater than 2 on the search path can be detached by giving their number to `detach`, but it is much safer to always use a name, for example `detach(lentils)` or `detach("lentils")`.* »

`detach()` sans argument détache « ce qui est en position 2 », quoi que ce soit. Si deux objets ont été attachés depuis, ce n'est plus celui qu'on croit. **Toujours nommer.**

4.3 La note encadrée du cours — la plus importante du chapitre

Note (§6.3.2, encadrée dans le cours). « *In R lists and data frames can only be attached at position 2 or above, and what is attached is a copy of the original object. You can alter the attached values via `assign`, but the original list or data frame is unchanged.* »

Trois faits, chacun décisif :

1. **Jamais en position 1** — l'espace de travail garde toujours la priorité. C'est ce qui rend le masquage possible.

2. **C'est une copie** — d'où l'invisibilité du `lentils$u <- v + w`.
3. **Modifier la copie via `assign` ne touche pas l'original** — la copie est un cul-de-sac.

● Concept 5 — La méthode de travail recommandée par le cours

MÉTHODE (§6.3.3).

« *A useful convention that allows you to work with many different problems comfortably together in the same workspace is* » — quatre règles, dans cet ordre :

1. « *gather together all variables for any well defined and separate problem in a data frame under a suitably informative name* » ;
2. « *when working with a problem attach the appropriate data frame at position 2, and use the workspace at level 1 for operational quantities and temporary variables* » ;
3. « *before leaving a problem, add any variables you wish to keep for future reference to the data frame using the `$` form of assignment, and then `detach()`* » ;
4. « *finally remove all unwanted variables from the workspace and keep it as clean of left-over temporary variables as possible* ».

EN CLAIR — C'EST LA SOLUTION DU PROBLÈME POSÉ EN FICHE 300.

Le §1.11 recommandait **un répertoire par analyse** pour que des noms comme `x` et `y` restent utilisables. Le §6.3.3 propose la solution **à l'intérieur d'une même session** : « *In this way it is quite simple to work with many problems in the same directory, all of which have variables named `x`, `y` and `z`* ».

Le data frame joue le rôle de **conteneur nommé**, et le chemin de recherche celui de **contexte courant**. Position 1 = le brouillon ; position 2 = le problème en cours.

« `attach()` is a generic function that allows not only directories and data frames to be attached to the search path, but other classes of object as well. In particular any object of mode "list" may be attached in the same way » (§6.3.4) — puisqu'un data frame **est** une liste, la généralité était acquise.

● Concept 6 — Le chemin de recherche

DÉFINITION (§6.3.5).

« The function `search` **shows the current search path** and so is **a very useful way to keep track of which data frames and lists (and packages) have been attached and detached.** »

Au départ :

```
search()
# [1] ".GlobalEnv" "Autoloads" "package:base"
```

« where `.GlobalEnv` **is the workspace** » (note 1 : « See the on-line help for `autoLoad` for the meaning of the second term »).

Après `attach(lentils)` :

```
search()
# [1] ".GlobalEnv" "lentils" "Autoloads" "package:base"

ls(2)
# [1] "u" "v" "w"
```

« and as we see `Ls` (or `objects`) **can be used to examine the contents of any position on the search path** ».

Après `detach("lentils")` :

```
search()
# [1] ".GlobalEnv" "Autoloads" "package:base"
```

EN CLAIR — COMMENT R TROUVE UN NOM.

Le chemin de recherche est une **liste ordonnée d'endroits où chercher**. R parcourt les positions **dans l'ordre** et s'arrête à la première correspondance. D'où :

- un objet de l'espace de travail (**position 1**) **masque** tout homonyme attaché ;
- un objet attaché masque un objet de même nom d'un paquet chargé plus loin ;
- `ls(n)` montre ce que contient la position `n`, ce qui permet de diagnostiquer un masquage.

C'est la mécanique complète que la fiche 310 reprendra du côté des **environnements** : le chemin de recherche **est** une chaîne d'environnements, chacun étant l'enclos du précédent.

Énoncé. `lentils` est un data frame de variables `u`, `v`, `w`. Un utilisateur exécute la séquence suivante et s'étonne du résultat final. Expliquer chaque étape.

```
attach(lentils)
u <- v + w
lentils$u <- v + w
u
detach()
u
```

Étape 1 — `attach(lentils)`. Le data frame — plus exactement **une copie** — est placé **en position 2** du chemin de recherche. `search()` montrerait `".GlobalEnv"` `"lentils"` `"AutoLoads"` `"package:base"`. Puisqu'aucun `u`, `v`, `w` n'existe en position 1, les trois sont « *available as variables from the data frame in their own right* ».

Étape 2 — `u <- v + w`. « **does not replace the component `u` of the data frame, but rather masks it with another variable `u` in the workspace at position 1** ». Une nouvelle variable `u` naît **dans** `.GlobalEnv`. La colonne du data frame est **intacte**, et **invisible** — puisque la position 1 est consultée d'abord.

Étape 3 — `lentils$u <- v + w`. Cette fois la colonne **est** modifiée. C'est la voie que le cours recommande : « *the simplest way is to **resort once again to the `$` notation*** ».

Étape 4 — `u` tapé à la console. Il rend le **`u` de la position 1**, celui de l'étape 2. Il se trouve qu'il a la même valeur — ce qui masque le problème plutôt que de le révéler.

Étape 5 — `detach()` . « *this statement **detaches from the search path the entity currently at position 2*** ». Ici c'est bien `lentils` , mais **par accident** : si l'utilisateur avait attaché autre chose entre-temps, il aurait détaché le mauvais objet. La forme sûre est `detach(lentils)` .

Étape 6 — `u` de nouveau. Il rend **toujours** le `u` de l'espace de travail : le `detach` a retiré la position 2, pas la position 1. **La variable temporaire survit au détachement** — c'est exactement le désordre que la règle 4 du §6.3.3 cherche à éviter : « *remove all unwanted variables from the workspace* ».

Étape 7 — ce qu'il aurait fallu faire.

```
attach(lentils)
tmp <- v + w           # un nom de travail EXPLICITE, pas le nom d'une colonne
lentils$u <- tmp       # modification permanente
detach(lentils)        # nomme, pas implicite
rm(tmp)                # nettoyage
attach(lentils)        # la nouvelle valeur devient visible
```

Étape 8 — la leçon. Le cours signale que la nouvelle valeur « ***is not visible until the data frame is detached and attached again*** ». Le cycle attacher / travailler / écrire par `$` / détacher / nettoyer est celui du §6.3.3, et **chacune de ses quatre règles répare l'un des pièges rencontrés ici**.

Comment reconnaître le type d'exercice

Le signal dans l'énoncé	Ce qu'il faut mobiliser
« stocker des objets de types différents »	une liste
« le résultat de la fonction »	c'est presque toujours une liste — <code>\$</code> ou <code>[[</code>
« je ne peux pas additionner le résultat »	vous avez <code>[</code> au lieu de <code>[[</code>
« le nom de la colonne est dans une variable »	<code>Lst[[x]]</code> , jamais <code>\$</code>
« <code>length()</code> me rend un nombre trop petit »	il compte les composantes de niveau supérieur
« <code>Lst\$coe</code> fonctionne pour <code>coefficients</code> »	appariement partiel — à éviter en script
« j'écris <code>Lst\$coe <- ...</code> et rien ne change »	l'appariement partiel ne joue pas en écriture
« ajouter une composante »	écrire hors des bornes : <code>Lst[5] <- list(...)</code>
« fusionner deux listes »	<code>c(listeA, listeB)</code>
« mon <code>dim</code> a disparu »	<code>c()</code> efface les attributs
« transformer une liste en tableau de données »	<code>as.data.frame()</code> — si les restrictions sont respectées
« colonnes de longueurs différentes »	interdit : « <i>must all have the same length</i> »
« <code>df[2]</code> ou <code>df[2,]</code> ? »	colonne contre ligne
« éviter d'écrire le nom du data frame partout »	<code>attach()</code> — avec toutes ses précautions
« ma modification n'apparaît pas »	<code>attach</code> attache une copie → <code>detach</code> puis <code>attach</code>
« ma variable en masque une autre »	le chemin de recherche — <code>search()</code> , <code>ls(2)</code>
« quel objet est visible ? »	le premier trouvé en parcourant le chemin

Comment résoudre ce type d'exercice

Protocole « exploiter un résultat de fonction » — 4 étapes.

1. `names(res)` — quelles composantes existent.

2. `str(res)` ou `length(res)` — combien, et de quelle nature.
3. Extraire avec `[[` ou `$`, jamais `[` — sauf si l'on veut vraiment une sous-liste.
4. Si le nom est calculé, `res[[nom]]` ; en script, écrire le nom **en entier** pour ne pas dépendre de l'appariement partiel.

Protocole « construire un data frame propre » — 4 étapes.

1. Vérifier **les trois restrictions**, en particulier **l'égalité des longueurs**.
2. `data.frame(...)` avec des **noms de variables explicites**, ou `as.data.frame()` sur une liste conforme.
3. Se rappeler qu'une **matrice** apporte **autant de variables qu'elle a de colonnes**.
4. Depuis un fichier : `read.table()` (fiche 307) — « *the simplest way to construct a data frame from scratch* ».

Protocole « travailler avec `attach()` sans se piéger » — 5 étapes.

1. **Attacher** le data frame du problème en cours : `attach(df)` .
2. **Ne jamais réutiliser un nom de colonne** pour une variable de travail — c'est ce qui crée le masquage.
3. Pour **modifier une colonne**, passer par `df$col <- ...` ; la nouvelle valeur ne sera **visible** qu'après un cycle `detach` / `attach` .
4. `detach(df)` **en nommant**, jamais `detach()` seul.
5. `rm()` les variables temporaires, et vérifier avec `search()` et `ls()` .

● Common mistakes

Erreur	Correction
Confondre <code>Lst[[1]]</code> et <code>Lst[1]</code>	l'objet contre une sous-liste d'un élément
Croire que <code>[]</code> sur une liste extrait	il rend toujours une liste
Utiliser <code>\$</code> avec un nom stocké dans une variable	<code>Lst[[x]]</code>
Compter sur l'appariement partiel en script	il est silencieux avec <code>\$</code> — et inactif en écriture
Croire <code>length()</code> égal au nombre de valeurs	il compte les composantes de niveau supérieur
Croire que mettre un objet dans une liste crée un lien	« <i>the components ... are copied ... the originals are not affected</i> »
Utiliser <code>c()</code> pour fusionner des matrices dans une liste	<code>c()</code> efface <code>dim</code>
Mettre des colonnes de longueurs différentes	« <i>must all have the same length</i> »
Croire qu'une matrice devient une colonne	elle fournit autant de variables qu'elle a de colonnes
Croire <code>df[2]</code> égal à <code>df[2,]</code>	colonne contre ligne
Croire <code>attach()</code> créer un lien vivant	c'est une copie
Écrire <code>u <- v+w</code> en croyant modifier la colonne	cela masque la colonne en position 1
Croire <code>df\$u <- ...</code> immédiatement visible	« <i>not visible until the data frame is detached and attached again</i> »
Utiliser <code>detach()</code> sans argument	il détache ce qui est en position 2 , quoi que ce soit
Croire pouvoir attacher en position 1	« <i>can only be attached at position 2 or above</i> »
Croire modifier l'original par <code>assign</code> sur la copie attachée	« <i>the original list or data frame is unchanged</i> »
Laisser traîner les variables de travail	règle 4 du §6.3.3 : nettoyer
Chercher pourquoi un nom « ne marche plus »	<code>search()</code> puis <code>ls(n)</code> — c'est un masquage

Ultimate Review

La liste. « *an ordered collection of objects known as its components* » · « *no particular need for the components to be of the same mode or type* » · une composante peut être **une fonction**, une matrice, une autre liste.

Trois accès. `Lst[[i]]` par numéro — toujours possible · `Lst[["nom"]]` par nom **calculable** · `Lst$nom` par nom **littéral**, non calculable. `Lst[[4]][1]` = premier élément de la 4^e composante.

** `[[` contre `[` — « *It is very important to distinguish* ». ** `Lst[[1]]` = **l'objet**, « *the name is **not** included* » · `Lst[1]` = **une sous-liste** d'une entrée, « *the names **are transferred** to the sublist* ».

`length()` compte « *the number of (top level) components* ».

Appariement partiel. « *may be abbreviated down to the minimum number of letters needed to identify them uniquely* » : `Lst$coe` pour `coefficients`, `Lst$cov` pour `covariance`. Silencieux avec `$`, **inactif en écriture**.

`names` **n'est qu'un attribut** « *like any other* » — d'autres structures peuvent en porter un.

Construire. `list(nom = objet, ...)` ; noms **librement choisis** ; omis → composantes **numérotées seulement** · « *the components ... **are copied** ... the originals are not affected* » · étendre par `Lst[5] <- list(...)` · `c()` sur des listes rend **une liste**, en joignant les composantes **en séquence** ; sur des vecteurs, « *all other attributes, such as `dim`, **are discarded*** ».

Le data frame. « *A data frame is a list with class "data.frame"* ». » **Trois restrictions** : (1) composantes = vecteurs (numériques, caractères, logiques), **facteurs, matrices numériques, listes**, ou **autres data frames** ; (2) matrices, listes et data frames fournissent **autant de variables qu'ils ont de colonnes, d'éléments ou de variables** ; (3) **toutes les longueurs égales** — et même nombre de lignes pour les matrices. « *may for many purposes be regarded as a matrix with columns possibly of differing modes and attributes* ». Attributs : `names` (variables) et `row.names` (cas). Le construire : `data.frame()`, `as.data.frame()`, ou `read.table()` — « *the simplest way ... from scratch* ».

`attach()` . Place l'objet **en position 2** · rend les composantes visibles **sous leur nom**, « *provided there are no variables ... in position 1* » · c'est **une fonction générique**, applicable à « *any object of mode "List"* ».

La note encadrée. « *In R lists and data frames **can only be attached at position 2 or above**, and what is attached is a copy of the original object. You can alter the attached values via `assign`, but the original list or data frame is unchanged.* »

Les deux pièges. `u <- v + w` « *does not replace the component ... but rather masks it* » · `df$u <- v + w` modifie bien, mais « *the new value ... is not visible until the data frame is detached and attached* »

again ».

`detach()` « *detaches from the search path **the entity currently at position 2*** » — « *it is much safer to always use a name* » : `detach(lentils)` ou `detach("lentils")` .

La méthode du §6.3.3 — quatre règles. (1) **regrouper** toutes les variables d'un problème dans un **data frame** ; (2) **attacher** en position 2, **travailler en position 1** ; (3) avant de partir, **ranger par \$** ce qu'on veut garder, puis `detach()` ; (4) **nettoyer** l'espace de travail. Ainsi « *it is quite simple to **work with many problems in the same directory, all of which have variables named x, y and z*** ».

Le chemin de recherche. `search()` l'affiche · au départ `.GlobalEnv` , `Autoloads` , `package:base` — « `.GlobalEnv` **is the workspace** » · après un `attach` , l'objet s'intercale **en position 2** · `ls(n)` (ou `objects(n)`) « *can be used to examine the contents of **any position** on the search path* » · R prend **la première correspondance** — d'où le masquage.

Active Recall

« An R list is **an object consisting of an ordered collection of objects known as its components**. **There is no particular need for the components to be of the same mode or type** — a list could consist of a numeric vector, a logical value, a matrix, a complex vector, a character array, **a function**, and so on. » (§6.1)

C'est la **structure récursive** de la fiche 303 : un vecteur atomique impose « *all of the same mode* », une liste ne l'impose pas. Et la *R Language Definition* (§2.1.2) précise que **les listes sont des vecteurs** — des « vecteurs génériques ».

`length()` **compte les composantes de niveau supérieur** : la liste `Lst` du cours a **4** composantes pour sept valeurs élémentaires.

Sur `Lst <- list(name = "Fred", wife = "Mary", no.children = 3, child.ages = c(4,7,9))` :

Écriture	Remarque
<code>Lst[[1]]</code>	par numéro — « <i>Components are always numbered</i> »
<code>Lst[["name"]]</code>	par nom, en chaîne
<code>Lst\$name</code>	par nom, littéral

« *This is especially useful **when the name of the component to be extracted is stored in another variable**, as in `x <- "name"; Lst[[x]]` » (§6.1).*

Quand le nom est calculé, `[[` est le seul correct : `$` n'évalue pas son index (fiche 302) et rendrait `NULL` sans erreur.

« *It is very important to distinguish `Lst[[1]]` from `Lst[1]` . `[[...]]` is the operator used to select a single element, whereas `[...]` is a general subscripting operator. Thus the former is the first object in the list, and if it is a named list the name is not included. The latter is a sublist ... consisting of the first entry only ; if it is a named list, the names are transferred to the sublist.* » (§6.1)

Pourquoi c'est « très important » : la plupart des fonctions de R rendent **des listes**.

`eigen(Sm)[1]` rend une **liste** contenant les valeurs propres — inutilisable telle quelle ;
`eigen(Sm)[[1]]` rend **le vecteur**. L'erreur qui suit ne parle jamais de crochets.

« *The components used to form the list are copied when forming the new list and the originals are not affected.* » (§6.2)

C'est la **sémantique de copie** de R, déjà rencontrée sous la forme du mécanisme ``*tmp*`` (fiche 302). Mettre `x` dans une liste ne crée **aucun lien** : modifier `Lst[[1]]` ne touche pas `x`, et modifier `x` ne touche pas la liste.

L'unique exception du langage reste **l'environnement**, « *not copied when passed to functions or used in assignments* » (fiche 303).

« When the concatenation function `c()` is given list arguments, the result is an object of mode `List` also, whose components are those of the argument lists joined together in sequence. » (§6.2.1)

```
list.ABC <- c(list.A, list.B, list.C)
```

C'est la précision annoncée par la note 1 du §2.1 : « With other than vector types of argument, such as list mode arguments, **the action of `c()` is rather different** ».

Et pour les vecteurs : « **all other attributes, such as `dim` attributes, are discarded** » — cohérent avec le §5.9 (fiche 305) et avec la règle générale de la fiche 303 (« coercion drops all attributes »).

« **A data frame is a list with class `"data.frame"`** . » (§6.3)

Les trois restrictions :

1. « The components must be **vectors (numeric, character, or logical), factors, numeric matrices, lists, or other data frames**. »
2. « **Matrices, lists, and data frames provide as many variables to the new data frame as they have columns, elements, or variables, respectively.** »
3. « **Vector structures ... must all have the same length, and matrix structures must all have the same number of rows.** »

La troisième est la contrainte : c'est l'égalité des longueurs qui autorise la lecture matricielle. La deuxième est la plus surprenante : une matrice y entre comme **plusieurs** variables.

« A data frame may **for many purposes** be regarded as **a matrix with columns possibly of differing modes and attributes**. It may be **displayed in matrix form**, and **its rows and columns extracted using matrix indexing conventions**. » (§6.3)

« **For many purposes** », pas tous. L'exception est donnée par la *R Language Definition* §3.4.3 : « **if a single index is supplied, it is interpreted as indexing the list of columns** — in that case **the drop argument is ignored, with a warning** ».

Donc `df[2]` est la deuxième colonne et `df[2, ]` la deuxième ligne. Avec deux indices, le data frame est matriciel ; avec un seul, il redevient la liste qu'il est.

« `attach()` places the data frame in the search path at position 2, and provided there are no variables `u`, `v` or `w` in position 1, `u`, `v` and `w` are available as variables from the data frame in their own right. » (§6.3.2)

La condition est le cœur du mécanisme : R parcourt le chemin de recherche dans l'ordre et prend la première correspondance. La position 1 — l'espace de travail — est toujours consultée en premier, donc tout homonyme y masque la colonne.

Et la note encadrée ajoute : « *lists and data frames can only be attached at position 2 or above, and what is attached is a copy of the original object* ».

Premier piège : « an assignment such as `u <- v + w` does not replace the component `u` of the data frame, but rather masks it with another variable `u` in the workspace at position 1 on the search path » (§6.3.2). Une nouvelle variable naît en position 1 ; la colonne est intacte et cachée.

Second piège : « To make a permanent change ... the simplest way is to resort once again to the `$` notation : `Lentils$u <- v + w`. However the new value of component `u` is not visible until the data frame is detached and attached again. »

La raison des deux est la même : ce qui est attaché est une copie. La modification touche l'original ; la copie en position 2, elle, ne change pas.

« More precisely, this statement detaches from the search path the entity currently at position 2. ... Entities at positions greater than 2 can be detached by giving their number to

`detach` , **but it is much safer to always use a name**, for example `detach(lentils)` or `detach("lentils")` . » (§6.3.2)

`detach()` détache **ce qui se trouve en position 2**, pas « *ce que j'ai attaché* ». Dès qu'un second objet a été attaché, les positions ont glissé et **on détache le mauvais objet** — sans erreur, puisque l'opération est parfaitement valide.

Nommer coûte trois caractères et supprime la classe entière de ce bug.

1. « **gather together all variables for any well defined and separate problem in a data frame** under a suitably informative name »
2. « **attach the appropriate data frame at position 2**, and **use the workspace at level 1 for operational quantities and temporary variables** »
3. « *before leaving a problem, add any variables you wish to keep ... to the data frame using the `$` form of assignment, and then `detach()`* »
4. « *finally remove all unwanted variables from the workspace* »

Le problème résolu est celui posé au §1.11 (fiche 300) : « *It is quite common for objects with names `x` and `y` to be created ... it can be quite hard to decide what they might be when several analyses have been conducted in the same directory.* »

La solution du chapitre 1 était **un répertoire par analyse** ; celle du chapitre 6 est **un data frame par problème**, ce qui permet — le cours le dit — de « **work with many problems in the same directory, all of which have variables named `x` , `y` and `z`** ».

```
search()
# [1] ".GlobalEnv" "Autoloads" "package:base"
```

« where `.GlobalEnv` **is the workspace** » (§6.3.5). Après `attach(lentils)` , `lentils` s'intercale **en position 2**.

Pour diagnostiquer, le cours donne l'outil : « **`ls` (or `objects`) can be used to examine the contents of any position on the search path** » — `ls(2)` rend ici `"u" "v" "w"` .

La méthode : `search()` pour voir l'ordre des positions, puis `ls(n)` pour voir laquelle contient le nom litigieux. R prend **la première correspondance** — celle de la position la plus basse.

« `attach()` **is a generic function** that allows not only directories and data frames to be attached to the search path, but **other classes of object as well**. In particular **any object of mode "list"** may be attached in the same way : `attach(any.old.list)` . » (§6.3.4)

C'était acquis d'avance, puisque « **a data frame is a list** with class `"data.frame"` ».

« **Anything that has been attached can be detached by** `detach` , **by position number or, preferably, by name**. » — la recommandation de nommer revient une seconde fois.

Flashcards

Question	Réponse
Qu'est-ce qu'une liste ?	Une collection ordonnée d'objets appelés composantes
Ses composantes doivent-elles être du même mode ?	Non
Une composante peut-elle être une fonction ?	Oui
Que rend <code>length(Lst)</code> ?	Le nombre de composantes de niveau supérieur
Accéder à la 3 ^e composante ?	<code>Lst[[3]]</code>
Par son nom, en chaîne ?	<code>Lst[["nom"]]</code>
Par son nom, littéralement ?	<code>Lst\$nom</code>
Le nom est dans une variable <code>x</code> ?	<code>Lst[[x]]</code>
Le 1 ^{er} élément de la 4 ^e composante ?	<code>Lst[[4]][1]</code>
Que rend <code>Lst[[1]]</code> ?	L'objet — le nom n'est pas inclus
Que rend <code>Lst[1]</code> ?	Une sous-liste d'une entrée, avec le nom
Classe de <code>Lst[1]</code> ?	<code>list</code>
Que dit le cours de cette distinction ?	« <i>It is very important to distinguish</i> »
Peut-on abréger les noms ?	Oui — jusqu'au minimum identifiant de façon unique
Abréviation de <code>Lst\$coefficients</code> ?	<code>Lst\$coe</code>
L'appariement partiel joue-t-il en écriture ?	Non
Le vecteur des noms, c'est quoi ?	Un attribut comme un autre
Comment créer une liste ?	<code>list(nom = objet, ...)</code>
Si les noms sont omis ?	Les composantes sont numérotées seulement
Que deviennent les objets d'origine ?	Ils sont copiés — non affectés
Comment étendre une liste ?	<code>Lst[5] <- list(...)</code>

Question	Réponse
Que rend <code>c()</code> sur des listes ?	Une liste , composantes jointes en séquence
Que devient <code>dim</code> avec <code>c()</code> ?	Il est écarté
Qu'est-ce qu'un data frame ?	Une liste de classe <code>"data.frame"</code>
Restriction 1 ?	Vecteurs, facteurs , matrices numériques, listes, data frames
Restriction 2 ?	Ils fournissent autant de variables que de colonnes
Restriction 3 ?	Toutes les longueurs égales
Un data frame se lit-il comme une matrice ?	Pour beaucoup d'usages , oui
L'exception ?	Un seul indice → les colonnes
<code>df[2]</code> ?	La deuxième colonne
<code>df[2,]</code> ?	La deuxième ligne
Ses deux attributs de nommage ?	<code>names</code> et <code>row.names</code>
Quelle structure imite-t-il ?	Le jeu de données SAS ou SPSS
Le créer à partir de vecteurs ?	<code>data.frame()</code>
À partir d'une liste conforme ?	<code>as.data.frame()</code>
La façon la plus simple, à partir de rien ?	<code>read.table()</code>
Que fait <code>attach(df)</code> ?	Il le place en position 2 du chemin
À quelle condition les colonnes sont-elles visibles ?	Qu'aucun homonyme n'existe en position 1
Peut-on attacher en position 1 ?	Non — « <i>position 2 or above</i> »
Qu'est-ce qui est attaché ?	Une copie
<code>assign</code> sur la copie modifie-t-il l'original ?	Non
Que fait <code>u <- v + w</code> après <code>attach</code> ?	Il masque la colonne, en position 1
Comment modifier vraiment la colonne ?	<code>df\$u <- v + w</code>

Question	Réponse
Le changement est-il visible aussitôt ?	Non — après <code>detach</code> puis <code>attach</code>
Que détache <code>detach()</code> sans argument ?	Ce qui est en position 2
La forme sûre ?	<code>detach(df)</code> ou <code>detach("df")</code>
<code>attach()</code> est-elle générique ?	Oui
Que peut-on attacher d'autre ?	Tout objet de mode <code>"list"</code>
Règle 1 de la méthode du §6.3.3 ?	Regrouper les variables d'un problème dans un data frame
Règle 2 ?	Attacher en 2 , travailler en 1
Règle 3 ?	Ranger par <code>\$</code> , puis <code>detach()</code>
Règle 4 ?	Nettoyer l'espace de travail
Quel problème cette méthode résout-elle ?	Plusieurs problèmes ayant tous des <code>x</code> , <code>y</code> , <code>z</code>
Quelle fonction affiche le chemin de recherche ?	<code>search()</code>
Son contenu initial ?	<code>.GlobalEnv</code> · <code>Autoloads</code> · <code>package:base</code>
Que désigne <code>.GlobalEnv</code> ?	L'espace de travail
Comment voir le contenu d'une position ?	<code>ls(n)</code> ou <code>objects(n)</code>
Quelle correspondance R retient-il ?	La première rencontrée